

Today 6:26 PM

I'm doing a uni lab on a pygame donkey kong game. Here's the repo: https://github.com/SETAPESU26/30_donkeykong (game.py and README.md). There are 4 tasks in total: the first one is a bug fix and the other 3 are features where I have to fill in empty functions. Let's focus on the bug first and do the features after.

The bug: barrels are supposed to take a ladder down only sometimes (around 3 out of 10 times), but right now almost every barrel takes the first ladder it reaches, so the lower platforms get flooded and the top one empties out.

Can you find the probability check in the Barrel class that controls this and fix it so barrels go down ~30% of the time instead of ~70%? Please only change what's needed, tell me which line was wrong and why, and don't rewrite the whole file.

I'm doing a uni lab on a pygame donkey kong game. Here's the repo: https://github.com/SETAPESU26/30_donkeykong (game.py and README.md). There are 4 tasks in total: the first one is a bug fix and the other 3 are features where I have to fill in empty functions. Let's focus on the bug first and do the features after.

The bug: barrels are supposed to take a ladder down only sometimes (around 3 out of 10 times), but right now almost every barrel takes the first ladder it reaches, so the lower platforms get flooded and the top one empties out.

Can you find the probability check in the Barrel class that controls this and fix it so barrels go down ~30% of the time instead of ~70%? Only change what's needed, tell me which line was wrong and why, and don't rewrite the whole file.

The bug is exactly in the `Barrel.update()` probability check.

Wrong line

In `game.py`, line 157:

 Python

```
if random.random() < 0.7:
```

Fix

Change only **0.7** to **0.3** :

</> Python

```
if random.random() < 0.3:
```

Why?

`random.random()` generates a random number from **0.0 up to 1.0**.

So:

</> Python

```
random.random() < 0.7
```

means approximately **70% probability** of taking the ladder.

You want barrels to take the ladder approximately **3 out of 10 times**, i.e. **30%**:

</> Python

```
random.random() < 0.3
```

So the relevant section becomes:

</> Python

```
if upper == self.plat and abs(self.pos.x - lx) < 3 and index not in self.skip:
    self.skip.add(index)
    if random.random() < 0.3:
        self.ladder = index
        self.pos.x = lx
```

Nothing else needs to be changed. [GitHub](#)

Bug: 0.7 → 0.3

Reason: the original code made ladder-taking a 70% event instead of a 30% event.

that worked, barrels take ladders way less now. moving on to the first feature, `theme_color(score)`. it's the empty function that's

called every frame in `draw_scene` like
`screen.fill(theme_color(score) or BG)`, where BG is the navy default (15, 15, 25).

it gets the score as an int and should return an (r, g, b) tuple for the background, or None to keep the default. I want the background to slowly get warmer as the score goes up, and stay navy at the start.

only fill in that function and don't touch the rest. just show me the function and where it goes. my current code is here:
<https://github.com/udaykeerthanreddykarnati/SE-PES1UG24CS217/blob/main/lab-4/game.py>

Put this directly in `game.py`, replacing the existing empty `theme_color()` function at **line 726**.
The rest of the file stays unchanged.  [GitHub](#)

 Python

```
def theme_color(score):  
    """Return an (r, g, b) background colour for the current score, or None for the default."""  
    if score < 500:  
        return None  
  
    warmth = min((score - 500) // 100, 10)  
  
    r = 15 + warmth * 8  
    g = 15 + warmth * 3  
    b = 25 - warmth * 2  
  
    return (r, g, b)
```

Where it goes

Replace only this:

 Python

```
def theme_color(score):  
    """Return an (r, g, b) background colour for the current score, or None for the default."""  
    pass
```

with the function above.

At scores below **500**, it returns `None`, so this line:

</> Python

```
screen.fill(theme_color(score) or BG)
```

uses the existing navy BG = (15, 15, 25) . As the score increases beyond 500, the red/green components gradually increase and blue decreases, making the background progressively warmer.

 GitHub

works, but the colour change is too subtle and takes too long to show up. can you make it start changing at around 100 points and get noticeably warmer by 800 or so, like going from navy to a dark orange/red? keep it as just the theme_color function, same rules, valid ints 0-255.

Replace only the existing theme_color() function with this:

</> Python

```
def theme_color(score):  
    """Return an (r, g, b) background colour for the current score, or None for the default."""  
    if score < 100:  
        return None  
  
    # Gradually transition from navy to dark orange/red between 100 and 800.  
    progress = min((score - 100) / 700, 1.0)  
  
    r = int(15 + 100 * progress)  
    g = int(15 + 25 * progress)  
    b = int(25 - 20 * progress)  
  
    return (r, g, b)
```

This gives roughly:

- **0–99:** default navy (15, 15, 25)
- **100:** starts changing
- **400:** visibly warmer
- **800+:** dark orange/red around (115, 40, 5)
- All RGB values remain valid **0–255**.

Today 8:24 PM

Task 2 is done, tested, committed, and pushed.

Now help me with Task 3 only.

Repo:

<https://github.com/udaykeerthanreddykarnati/SE-PES1UG24CS217>

File:

lab-4/game.py

Task 3 is the empty function:

```
def on_barrel_jumped(player, barrel):  
    """Called when the player clears a barrel; add a bonus effect  
    here."""  
    pass
```

Check the current game.py and README.md in the repo.

The function is called once when the player jumps over a barrel, immediately after the 100-point bonus.

I can ONLY modify this function. Do not change any other function or add new global variables/classes.

Give me:

1. The exact function I should put there.
2. What it does.
3. Confirm that it actually works with the existing game code and doesn't require changes elsewhere.

Keep it simple and suitable for the lab.

I checked the current `game.py` and `README.md`. Task 3 is called exactly where you described: after the 100-point bonus, and its return value is ignored.  [GitHub](#) +1

Since you can only modify this function and cannot add a global label/sprite system, the cleanest self-contained effect is a short **bonus sound**.

1. Replace the empty function with

```
def on_barrel_jumped(player, barrel):
    """Called when the player clears a barrel; add a bonus effect here."""
    import math
    from array import array

    mixer_info = pygame.mixer.get_init()
    if mixer_info is None:
        return

    frequency, _, channels = mixer_info
    duration = 0.08
    tone = array("h")

    for i in range(int(frequency * duration)):
        sample = int(5000 * math.sin(2 * math.pi * 880 * i / frequency))
        if channels == 1:
            tone.append(sample)
        else:
            tone.extend([sample, sample])
```

2. What it does

Every time the player successfully jumps over a barrel:

- The function is called **once**.
- It generates a very short **880 Hz beep**.
- The beep lasts about **0.08 seconds**.
- It plays immediately as the barrel-jump bonus happens.
- It doesn't change the score or gameplay logic.

The existing call is already:

 Python

```
score += int(100 * (score_multiplier(score) or 1))
on_barrel_jumped(player, barrel)
```

so the sound occurs immediately after the bonus.  GitHub

3. Does it work without changes elsewhere?

Yes. No other function, global variable, class, import, or file needs to be changed.

The imports are deliberately **inside the function**, so you're still modifying only

`on_barrel_jumped()`. The existing `pygame.init()` in `main()` initializes the mixer, and the function also safely does nothing if the mixer isn't available.  GitHub

This is also consistent with the README's Task 3 allowance of a sound as the side effect.  GitHub

Task 3 is done and pushed.

Now help me with Task 4.

Repo:

<https://github.com/udaykeerthanreddykarnati/SE-PES1UG24CS217>

File:

lab-4/game.py

Task 4 is:

```
def score_multiplier(score):  
    """Return a multiplier applied to points earned from clearing a  
    barrel, or None for the default 1x."""  
    pass
```

Check the current game.py and README.md.

The function is used like this:

```
score += int(100 * (score_multiplier(score) or 1))
```

Give me a simple implementation based on the README. I can only change this function. Don't change anything else.

Also explain the threshold you choose and what happens at different scores.

Based on the README, Task 4 is intended to **double the barrel-jump bonus once the player reaches a chosen score threshold**.  GitHub

A simple implementation is to use **500 points** as the threshold:

 Python

```
def score_multiplier(score):  
    """Return a multiplier applied to points earned from clearing a barrel, or None for the default 1x.  
    if score >= 500:
```

```
    return 2  
    return None
```

What happens at different scores

Current score	Multiplier	Barrel jump points
0–499	None → 1×	100
500+	2 → 2×	200
700	2 → 2×	200
1000	2 → 2×	200

The important detail is that `score_multiplier(score)` receives the **score before the new 100-point bonus is added**. So at exactly `500`, that barrel jump gives **200 points**, taking the score to `700`.

The existing line handles the default correctly:

 Python

```
score += int(100 * (score_multiplier(score) or 1))
```

When the function returns `None`, `None` or `1` becomes `1`. When it returns `2`, the bonus becomes $100 \times 2 = 200$.  [GitHub](#)

Only replace the empty function. No other changes are required.